

Hermes Web App Navigator — Full Project Plan

1. Project Summary

Hermes Web App Navigator is a Burp-assisted pentest companion that helps a tester understand a web application during an authorized assessment.

The goal is not to replace manual testing or blindly exploit applications. The goal is to help the tester build a live mental model of the target, discover hidden API surfaces, understand frontend-to-backend relationships, map traffic to OWASP-style testing ideas, and track what still needs to be checked.

Hermes should act like a guided pentest assistant:

- “What does this app seem to do?”
 - “Which APIs does the frontend actually call?”
 - “What domains are part of the real attack surface?”
 - “Which workflows are security-sensitive?”
 - “Which endpoints look high-value?”
 - “What should I test next in Repeater?”
 - “What have I already tested?”
 - “Where are likely IDOR, role, tenant, workflow, upload, auth, or business logic issues?”
-

2. Core Problem

During web application pentests, it is common to get lost in large applications.

Problems Hermes should solve:

1. The frontend and backend may use different domains.
2. The app may call APIs that are not obvious from the visible URL.
3. Important workflows may be spread across many endpoints.
4. Role boundaries may be unclear.
5. Object relationships may be unclear.
6. It is easy to forget which tests have already been performed.
7. Scanners often miss business logic and authorization flaws.
8. A tester may not immediately understand what the website is for.
9. Repeater testing can become unstructured.
10. Notes and evidence can become scattered.

Hermes should help organize the assessment while the tester works.

3. Design Philosophy

Hermes should be:

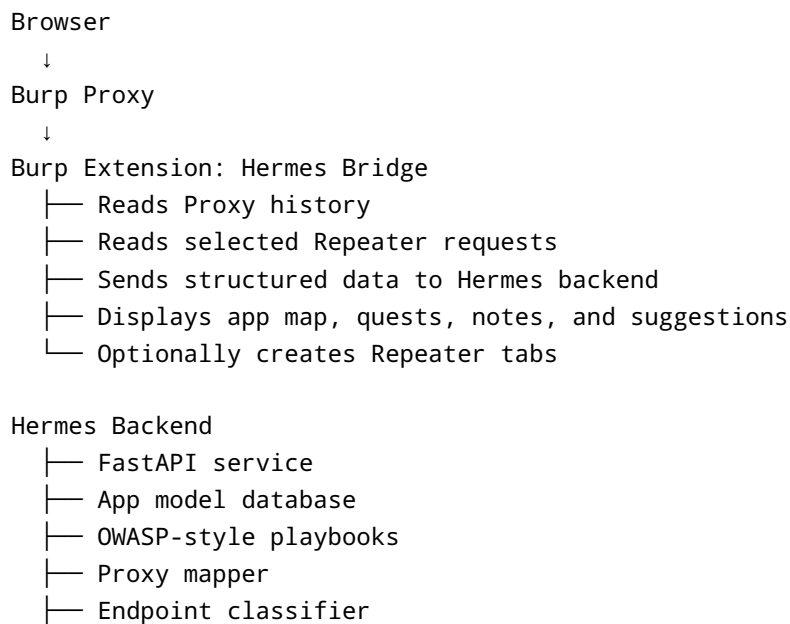
- **Human-in-the-loop**
- **Passive-first**
- **Architecture agnostic**
- **Burp-integrated**
- **Provider-agnostic for LLMs**
- **Scope-aware**
- **Evidence-driven**
- **Useful even without full automation**
- **Focused on understanding and guidance, not blind exploitation**

Hermes should not say “this is vulnerable” unless there is direct evidence.

It should say things like:

- “This looks like a possible IDOR candidate.”
- “This endpoint appears to use a tenant/project ID.”
- “Replay this as a lower-privileged user to confirm authorization.”
- “This JavaScript file references an admin route that has not appeared in Proxy yet.”
- “You have tested the frontend domain, but most authenticated API traffic is going to a different host.”

4. High-Level Architecture



- └─ Workflow builder
- └─ Quest generator
- └─ LLM provider abstraction
- └─ Evidence tracker

LLM Provider

- └─ OpenAI-compatible API
- └─ Ollama
- └─ LM Studio
- └─ Hermes local endpoint
- └─ Mock provider for testing

5. Architecture-Agnostic Requirement

Hermes must run on both x86_64/amd64 and ARM64.

Supported targets:

```
linux/amd64
linux/arm64
Apple Silicon through Docker Desktop
x86 laptops/workstations
ARM64 servers or SBCs where practical
```

Design rules:

1. Backend runs in Docker.
2. Docker image supports `linux/amd64` and `linux/arm64`.
3. Burp extension is a Java or Kotlin `.jar`.
4. No architecture-specific binaries in the core system.
5. Avoid dependencies that require difficult native compilation.
6. Avoid hardcoded paths.
7. Use environment variables for config.
8. Keep LLM provider swappable.
9. Use JSON or SQLite for first storage version.
10. Burp and Hermes communicate over HTTP.

Example multi-arch build command:

```
docker buildx build
  --platform linux/amd64,linux/arm64
```

```
-t hermes-burp-assistant:latest
.
```

Recommended backend Dockerfile:

```
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000

CMD ["uvicorn", "hermes_api.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

6. Main Components

6.1 Burp Extension

The Burp extension is the bridge between Burp and Hermes.

Responsibilities:

- Read selected HTTP request/response from Repeater.
- Read Proxy history.
- Read selected Proxy entries.
- Send sanitized request/response data to Hermes backend.
- Display Hermes results inside Burp.
- Create Repeater tabs from suggested tests.
- Mark checklist items as complete.
- Store local project/session metadata.
- Respect Burp target scope.

Right-click menu ideas:

```
Send to Hermes: Analyze request
Send to Hermes: Update app model
Send to Hermes: Suggest OWASP checks
Send to Hermes: Create Repeater ideas
```

```
Send to Hermes: Add to evidence journal
Send to Hermes: Mark workflow tested
```

Burp tab ideas:

```
Hermes App Map
Hermes Proxy Map
Hermes Quests
Hermes Endpoint Notes
Hermes Suggested Tests
Hermes Evidence Journal
Hermes Settings
```

6.2 Hermes Backend

The backend contains the actual logic.

Responsibilities:

- Receive HTTP data from Burp.
- Parse requests and responses.
- Extract domains, endpoints, parameters, JSON keys, cookies, auth headers, CORS headers, and referer/origin relationships.
- Cluster endpoints.
- Infer app features.
- Infer workflows.
- Build an app mental model.
- Match traffic to OWASP-style playbooks.
- Generate manual Repeater test ideas.
- Generate testing quests.
- Track evidence.
- Store app model.
- Talk to an LLM provider.

Backend stack:

```
Python 3.12
FastAPI
Pydantic
SQLite or JSON
PyYAML
```

```
HTTPX or Requests
Uvicorn
```

Avoid for MVP:

```
Playwright
Pandas
NumPy
heavy local ML libraries
browser automation
architecture-specific binaries
```

6.3 LLM Provider Abstraction

Hermes should not depend on one model.

Provider interface:

```
class LLMProvider:
    def complete(self, prompt: str) -> str:
        raise NotImplementedError
```

Provider modules:

```
providers/
  openai_compatible.py
  ollama.py
  lmstudio.py
  hermes_local.py
  mock.py
```

Environment config:

```
HERMES_PROVIDER=openai_compatible
HERMES_BASE_URL=http://localhost:11434/v1
HERMES_MODEL=some-model-name
HERMES_API_KEY=optional
```

The Burp extension should not care what model is running. It should only talk to:

```
POST http://localhost:8000/analyze-request
POST http://localhost:8000/proxy/import
GET http://localhost:8000/app-summary/{scope_name}
```

7. MVP Development Strategy

Do not start with the Burp extension.

Start with the backend.

MVP 1: Manual Request Analyzer

The tester manually pastes:

- raw HTTP request
- raw HTTP response
- known role
- app notes
- scope name

Hermes returns:

- endpoint summary
- feature
- workflow
- business objects
- sensitive parameters
- possible roles
- likely OWASP focus areas
- manual Repeater tests
- evidence to collect
- app model updates

This proves the reasoning layer works before building Burp integration.

MVP 2: Application Mental Model

Hermes stores the app model and can summarize:

- app type
- observed domains
- observed features
- business objects

- workflows
 - roles
 - sensitive actions
 - high-value endpoints
 - untested areas
-

MVP 3: Quest System

Hermes generates checklist-style pentest quests:

- Tenant Boundary Quest
 - Role Escalation Quest
 - IDOR Quest
 - Auth/Session Quest
 - Upload Quest
 - Checkout Logic Quest
 - Admin Surface Quest
 - GraphQL Quest
 - API Discovery Quest
-

MVP 4: Proxy History Import

Hermes reads Proxy history through the Burp extension and maps:

- frontend hosts
 - API hosts
 - auth hosts
 - upload/download hosts
 - GraphQL endpoints
 - WebSocket endpoints
 - third-party services
 - JavaScript-discovered routes
-

MVP 5: Burp Repeater Integration

Hermes can:

- analyze selected Repeater request
- suggest mutations
- create new Repeater tabs
- compare responses
- save evidence
- mark tests complete

MVP 6: BChecks / Custom Scan Check Generation

Hermes can generate reusable custom checks from confirmed patterns, but only after the tester validates the logic manually.

8. Proxy Mapping Feature

This is one of the most important features.

Modern apps often look like this:

```
https://app.example.com
https://api.example.com
https://auth.example.com
https://files.example-cdn.com
https://graphql.example.com
https://payments.vendor.com
```

The visible frontend may not be where the real attack surface is.

Hermes should passively map:

```
Frontend pages → API calls → business workflows → security test ideas
```

8.1 Proxy Mapping Flow

```

Burp Proxy History
↓
Passive Domain Extractor
↓
Frontend/API Relationship Mapper
↓
Endpoint Classifier
↓
Workflow Builder
↓
Hermes App Map
↓
Suggested Tests and Quests
```

8.2 Data Extracted From Proxy Traffic

For each request:

```
method
scheme
host
port
path
query parameters
content type
body shape
cookies present
authorization header present
origin header
referrer header
sec-fetch-site
sec-fetch-mode
user agent
websocket upgrade header
```

For each response:

```
status code
content type
body shape
CORS headers
set-cookie headers
redirects
cache headers
security headers
JSON keys
error messages
response length
```

8.3 Domain Classification

Hermes should classify domains as:

```
frontend
likely_first_party_api
auth
asset
```

```
file_storage
graphql
websocket
third_party
unknown
out_of_scope
```

Likely first-party signals:

```
same registrable domain
shared cookies
Authorization header sent
Origin points to frontend
Referer points to frontend
CORS allows frontend origin
API returns app-specific data
hostnames share naming pattern
```

Likely third-party signals:

```
analytics provider
payment provider
error tracking provider
font/static asset provider
no app-specific data
telemetry-only paths
no auth token sent
```

Hermes should not automatically exclude third-party domains. It should separate them and let the tester decide what is in scope.

8.4 Domain Relationship Model

Example:

```
{
  "frontend_hosts": ["app.example.com"],
  "api_hosts": ["api.example.com", "auth.example.com"],
  "asset_hosts": ["cdn.example.com"],
  "third_party_hosts": ["stripe.com", "sentry.io"],
  "relationships": [
    {
      "source": "app.example.com",
```

```
    "target": "api.example.com",
    "relationship_type": "frontend_calls_api",
    "evidence": [
      "Referer: https://app.example.com/dashboard",
      "Origin: https://app.example.com",
      "CORS allows app.example.com"
    ],
    "confidence": "high"
  }
]
```

8.5 Frontend-to-API Bridge

Hermes should connect frontend actions to backend calls.

Example:

```
Observed page:
https://app.example.com/projects/4812/settings

Observed API calls:
GET https://api.example.com/projects/4812
GET https://api.example.com/projects/4812/members
POST https://api.example.com/projects/4812/invitations
PATCH https://api.example.com/projects/4812/roles

Hermes inference:
This is the project settings/member management workflow.

Suggested checks:
- project IDOR
- tenant boundary bypass
- member-to-admin role escalation
- invite abuse
- role tampering
- replay admin actions as lower-privileged user
```

9. JavaScript Route Discovery

Hermes should inspect JavaScript responses from Proxy history.

For JavaScript files, search for:

```
absolute API URLs
relative API paths
GraphQL endpoints
WebSocket URLs
admin routes
feature flags
role names
upload paths
download paths
export paths
debug routes
environment names
```

Example output:

```
JavaScript route discoveries:

From /assets/app.8f3a1.js:
- /api/admin/users
- /api/billing/plans
- /api/projects/:id/export
- /api/internal/feature-flags
- wss://events.example.com/socket
```

Routes discovered in JavaScript but not seen in Proxy should be marked:

```
status: discovered_not_observed
suggested_check: forced browsing / role access review
```

10. App Mental Model

Hermes should maintain a live model of the app.

Example output:

```
Current understanding:

This appears to be a multi-tenant SaaS project management application.
```

Main features:

- authentication
- project dashboard
- team invitations
- role management
- billing
- file export

Important objects:

- user
- project
- membership
- invitation
- invoice
- billing plan

Likely roles:

- owner
- admin
- member
- viewer

Highest-value tests:

1. Project IDOR
2. Tenant boundary bypass
3. Member-to-admin role escalation
4. Invite token abuse
5. Billing plan tampering
6. Export authorization bypass

Hermes should be able to answer:

What does this app seem to do?
What are the main workflows?
Which domains are likely first-party APIs?
Which endpoints look high-value?
Which objects are security-sensitive?
What has not been tested?
What should I test next?

11. OWASP-Style Playbooks

Hermes should not rely only on LLM guesses.

It should use structured playbooks.

Initial playbooks:

1. Broken Access Control / IDOR
2. Authentication and Session Testing
3. Injection
4. File Upload Testing
5. Business Logic / Workflow Bypass
6. Tenant Isolation
7. Role Escalation
8. GraphQL Testing
9. API Discovery
10. Security Misconfiguration

Example playbook:

```
id: broken_access_control_idor
name: IDOR / Object Authorization Check
owasp_category: Broken Access Control

signals:
  - numeric_id_in_path
  - uuid_in_path
  - object_id_parameter
  - account_id
  - user_id
  - project_id
  - invoice_id
  - organization_id
  - tenant_id

use_when:
  - endpoint_returns_object_data
  - endpoint_modifies_object
  - endpoint_downloads_file
  - endpoint_exports_data

tests:
  - replay_as_lower_privileged_user
  - swap_object_id
  - remove_authorization_header
  - compare_cross_tenant_access
  - test_write_action_as_wrong_user
```

```
evidence:
  - unauthorized_200_response
  - sensitive_fields_present
  - object_owner_mismatch
  - write_action_succeeds
  - file_download_succeeds

false_positive_checks:
  - object_is_public
  - user_has_shared_access
  - endpoint_returns_only_public_metadata
```

12. Quest System

Quests are checklist-style test paths.

They help the tester stay organized.

12.1 Tenant Boundary Quest

Tenant Boundary Quest

- [] Identify tenant/org/project/account IDs
- [] Create or obtain User A and User B
- [] Confirm users belong to different tenants or projects
- [] Capture read requests from User A
- [] Replay read requests as User B
- [] Swap object IDs between tenants
- [] Test write requests across tenants
- [] Test invite/member APIs across tenants
- [] Test export/download APIs across tenants
- [] Save evidence for any unauthorized access

12.2 Role Escalation Quest

Role Escalation Quest

- [] Identify available roles
- [] Capture admin/owner action
- [] Replay action as lower-privileged user
- [] Modify role fields in request body
- [] Try hidden role values

- ☐ Check frontend-only restrictions
- ☐ Confirm actual server-side effect
- ☐ Save request/response evidence

12.3 API Discovery Quest

API Discovery Quest

- ☐ Review Proxy hosts
- ☐ Identify frontend domain
- ☐ Identify likely first-party API domains
- ☐ Review JavaScript files for hidden routes
- ☐ Check GraphQL endpoints
- ☐ Check WebSocket endpoints
- ☐ Check discovered but unobserved routes
- ☐ Check admin/internal-looking paths
- ☐ Confirm scope before testing extra domains

12.4 Business Logic Quest

Business Logic Quest

- ☐ Identify intended workflow sequence
- ☐ Repeat important steps
- ☐ Skip optional-looking steps
- ☐ Perform steps out of order
- ☐ Replay completed actions
- ☐ Change state-related parameters
- ☐ Test low-privileged user behavior
- ☐ Compare frontend restrictions against API behavior

12.5 Upload Quest

Upload Quest

- ☐ Identify upload endpoints
- ☐ Identify accepted file types
- ☐ Check where uploaded files are stored
- ☐ Check who can access uploaded files

- [] Test authorization on download URLs
- [] Check metadata exposure
- [] Check stored content rendering
- [] Check file replacement or overwrite behavior

13. API Contracts

13.1 Analyze Single Request

`POST /analyze-request`

Request:

```
{
  "scope_name": "Example App",
  "known_role": "member",
  "request": "RAW HTTP REQUEST HERE",
  "response": "RAW HTTP RESPONSE HERE",
  "tester_notes": "Captured after logging in as a normal user.",
  "source_tool": "repeater"
}
```

Response:

```
{
  "endpoint_summary": "This endpoint appears to retrieve project details by project ID.",
  "feature": "Project management",
  "workflow": "View project details",
  "business_objects": ["project", "user", "membership"],
  "sensitive_parameters": ["project_id"],
  "likely_roles": ["owner", "admin", "member", "viewer"],
  "likely_risks": [
    {
      "category": "Broken Access Control",
      "reason": "The endpoint uses a direct project identifier.",
      "confidence": "medium"
    }
  ],
  "suggested_tests": [
    {
      "name": "Project IDOR check",
    }
  ]
}
```

```

    "reason": "Project ID appears in the URL path.",
    "manual_steps": [
      "Capture the same request as User A.",
      "Replay it as User B.",
      "Swap project IDs.",
      "Compare response data and authorization behavior."
    ],
    "repeater_mutations": [
      "Change project ID in path",
      "Replay using lower-privileged session",
      "Remove Authorization header"
    ],
    "evidence_to_collect": [
      "status code",
      "response body",
      "object owner",
      "sensitive fields"
    ]
  }
],
"model_updates": [
  {
    "type": "business_object",
    "name": "project",
    "evidence": "Path contains /projects/{id}"
  }
]
}

```

13.2 Import Proxy History

`POST /proxy/import`

Request:

```

{
  "scope_name": "Example App",
  "items": [
    {
      "request": "RAW HTTP REQUEST",
      "response": "RAW HTTP RESPONSE",
      "tool": "proxy",
      "timestamp": "2026-05-31T20:00:00+09:00"
    }
  ]
}

```

```
]
}
```

Response:

```
{
  "imported": 120,
  "domains_discovered": 8,
  "api_hosts_discovered": 3,
  "new_endpoints": 47,
  "javascript_routes_discovered": 12,
  "high_value_findings": [
    "api.example.com contains authenticated project endpoints",
    "auth.example.com contains session and MFA endpoints",
    "JavaScript references /api/admin/users but no request has been observed"
  ]
}
```

13.3 Summarize App

```
GET /app-summary/{scope_name}
```

Response:

```
{
  "app_summary":
    "This appears to be a multi-tenant SaaS application for project and team management.",
  "frontend_hosts": ["app.example.com"],
  "likely_api_hosts": ["api.example.com", "auth.example.com"],
  "third_party_hosts": ["stripe.com", "sentry.io"],
  "observed_features": [
    "authentication",
    "project management",
    "member invitations",
    "billing",
    "file export"
  ],
  "important_objects": [
    "user",
    "project",
    "membership",
  ]
}
```

```
    "invitation",
    "invoice"
  ],
  "likely_roles": [
    "owner",
    "admin",
    "member",
    "viewer"
  ],
  "highest_value_tests": [
    "project IDOR",
    "tenant boundary bypass",
    "role escalation",
    "invite token abuse",
    "billing tampering"
  ],
  "untested_areas": [
    "export endpoints",
    "admin routes discovered in JavaScript",
    "MFA challenge endpoints"
  ]
}
```

13.4 Generate Quests

`POST /generate-quests`

Request:

```
{
  "scope_name": "Example App",
  "quest_type": "auto"
}
```

Response:

```
{
  "quests": [
    {
      "name": "Tenant Boundary Quest",
      "reason": "Multiple endpoints use project_id and membership objects.",
      "tasks": [
```

```
{
  "description": "Create or identify two users in different projects.",
  "status": "todo"
},
{
  "description": "Replay project read requests across users.",
  "status": "todo"
}
]
}
]
```

13.5 Save Evidence

`POST /evidence`

Request:

```
{
  "scope_name": "Example App",
  "title": "Possible project IDOR",
  "category": "Broken Access Control",
  "endpoint": "GET /api/projects/{id}",
  "request": "RAW HTTP REQUEST",
  "response": "RAW HTTP RESPONSE",
  "notes": "User B could access User A's project details.",
  "confidence": "high"
}
```

Response:

```
{
  "saved": true,
  "evidence_id": "ev_001"
}
```

14. Data Model

For MVP, SQLite is preferred.

apps

```
id
name
base_url
description
created_at
updated_at
```

domains

```
id
app_id
host
classification
confidence
in_scope
first_seen
last_seen
notes
```

domain_relationships

```
id
app_id
source_host
target_host
relationship_type
evidence
confidence
```

endpoints

```
id
app_id
method
host
path
normalized_path
query_params
body_shape
content_type
auth_observed
```

```
source_tool
first_seen
last_seen
feature
workflow
risk_score
tested_status
notes
```

business_objects

```
id
app_id
name
example_parameters
example_values
related_endpoints
sensitivity
confidence
```

workflows

```
id
app_id
name
steps
related_endpoints
likely_roles
risk_notes
confidence
```

discovered_routes

```
id
app_id
source_file
route
route_type
host
observed_in_proxy
tested_status
notes
```

quests

```
id
app_id
name
reason
status
created_at
updated_at
```

quest_tasks

```
id
quest_id
description
status
related_endpoint
evidence_id
notes
```

evidence

```
id
app_id
title
category
endpoint
request
response
notes
confidence
created_at
```

15. Risk Scoring

Hermes should assign priority to endpoints.

Increase risk score for:

```
Authorization header present
session cookie present
```

```
object IDs
user IDs
account IDs
org IDs
tenant IDs
project IDs
invoice/order IDs
admin path
billing/payment path
file upload/download
export/report endpoints
role or permission fields
GraphQL endpoint
state-changing method: POST, PUT, PATCH, DELETE
CORS relationship with frontend
sensitive JSON fields
JavaScript-discovered but untested routes
```

Lower risk score for:

```
static assets
images
fonts
CSS
analytics beacons
known third-party telemetry
public marketing pages
favicon
health checks with no sensitive data
```

Example output:

```
High priority:
- POST /api/projects/{id}/members
- PATCH /api/projects/{id}/roles
- GET /api/projects/{id}/export
- POST /api/billing/subscription
- POST /api/invitations

Medium priority:
- GET /api/users/me
- GET /api/projects
- GET /api/settings

Low priority:
```

- GET /assets/app.js
- GET /favicon.ico
- GET /styles.css

16. Hermes Prompt Template

You are assisting with an authorized web application penetration test.

Your job is not to exploit automatically.

Your job is to help the tester understand the application, identify likely workflows, map endpoints to security-relevant behavior, and suggest safe manual tests.

Given:

- one HTTP request
- one HTTP response
- known tester notes
- current app model if available
- OWASP-style testing playbooks

Return structured JSON with:

1. endpoint summary
2. likely feature
3. likely workflow
4. business objects involved
5. sensitive parameters
6. likely required role
7. possible security assumptions
8. suggested manual Repeater tests
9. evidence to collect
10. updates to the app model

Rules:

- Do not claim a vulnerability exists without direct evidence.
- Use confidence levels.
- Prefer practical pentester guidance.
- Do not generate destructive tests.
- Do not test out-of-scope hosts.
- Ask for human confirmation before active testing.
- Treat model output as suggestions, not proof.

17. Repository Structure

```
hermes-burp-assistant/  
  README.md  
  docker-compose.yml  
  Dockerfile  
  requirements.txt  
  .env.example  
  
hermes_api/  
  __init__.py  
  main.py  
  config.py  
  models.py  
  storage.py  
  parser.py  
  analyzer.py  
  proxy_mapper.py  
  endpoint_classifier.py  
  workflow_builder.py  
  quest_generator.py  
  evidence.py  
  
hermes_api/providers/  
  __init__.py  
  base.py  
  openai_compatible.py  
  ollama.py  
  lmstudio.py  
  mock.py  
  
hermes_api/playbooks/  
  broken_access_control.yaml  
  authentication.yaml  
  injection.yaml  
  upload.yaml  
  business_logic.yaml  
  tenant_isolation.yaml  
  role_escalation.yaml  
  graphql.yaml  
  api_discovery.yaml  
  misconfiguration.yaml  
  
hermes_api/prompts/  
  request_analyzer.txt  
  app_summarizer.txt
```

```
quest_generator.txt
proxy_mapper.txt

hermes_api/data/
hermes.sqlite

burp-extension/
build.gradle
settings.gradle
src/main/java/
com/hermes/BurpExtension.java
com/hermes/HermesClient.java
com/hermes/HermesTab.java
com/hermes/ContextMenuProvider.java
com/hermes/ProxyImporter.java
com/hermes/RepeaterActions.java

tests/
test_parser.py
test_proxy_mapper.py
test_endpoint_classifier.py
test_quest_generator.py
```

18. Build Order

Phase 1: Backend Skeleton

Goal:

FastAPI backend runs locally and in Docker.

Tasks:

```
[ ] Create repo
[ ] Add FastAPI app
[ ] Add /health endpoint
[ ] Add Dockerfile
[ ] Add docker-compose.yml
[ ] Add .env.example
[ ] Add provider abstraction
[ ] Add mock provider
```

Success:

```
docker compose up --build
curl http://localhost:8000/health
```

Phase 2: Manual Request Analyzer

Goal:

Paste one request/response and get a useful analysis.

Tasks:

- [] Add /analyze-request endpoint
- [] Parse raw HTTP request
- [] Parse raw HTTP response
- [] Extract method, host, path, params, headers, body shape
- [] Send structured prompt to mock/real LLM provider
- [] Return structured JSON
- [] Add app model update logic

Success:

User can paste a Repeater request/response and Hermes returns likely feature, workflow, risks, and manual tests.

Phase 3: OWASP Playbooks

Goal:

Hermes uses structured playbooks instead of pure LLM guessing.

Tasks:

- [] Create YAML playbook format
- [] Add Broken Access Control playbook
- [] Add Auth/Session playbook
- [] Add Injection playbook
- [] Add Upload playbook

- ☐ Add Business Logic playbook
- ☐ Add playbook matcher

Success:

Hermes can identify playbook matches from request/response features.

Phase 4: App Model

Goal:

Hermes remembers what it has seen.

Tasks:

- ☐ Add SQLite database
- ☐ Store domains
- ☐ Store endpoints
- ☐ Store business objects
- ☐ Store workflows
- ☐ Store test reminders
- ☐ Add /app-summary endpoint

Success:

After analyzing 10 requests, Hermes can summarize what the app appears to do.

Phase 5: Quest Generator

Goal:

Hermes creates testing checklists from the app model.

Tasks:

- ☐ Add /generate-quests endpoint
- ☐ Create quest templates

- [] Link quests to endpoints and objects
- [] Track task status
- [] Add /quests endpoint
- [] Add /quests/{id}/complete-task endpoint

Success:

Hermes generates relevant quests based on observed traffic.

Phase 6: Proxy Import

Goal:

Hermes can process many Proxy history items.

Tasks:

- [] Add /proxy/import endpoint
- [] Batch process proxy items
- [] Extract domains
- [] Extract frontend/API relationships
- [] Detect likely first-party APIs
- [] Detect third-party services
- [] Cluster endpoints
- [] Add discovered routes

Success:

Hermes can summarize frontend hosts, API hosts, auth hosts, and high-value endpoint groups.

Phase 7: JavaScript Route Discovery

Goal:

Hermes finds hidden API routes in frontend JavaScript.

Tasks:

- ☐ Detect JavaScript responses
- ☐ Extract URLs and route-looking strings
- ☐ Normalize discovered routes
- ☐ Mark routes as observed or unobserved
- ☐ Add discovered routes to app model
- ☐ Prioritize admin/internal/export/upload routes

Success:

Hermes reports routes found in JS but not yet seen in Proxy.

Phase 8: Burp Extension MVP

Goal:

Burp can send data to Hermes.

Tasks:

- ☐ Create Java/Kotlin Burp extension
- ☐ Add Hermes settings tab
- ☐ Add backend URL config
- ☐ Add context menu for selected request
- ☐ Send selected request/response to /analyze-request
- ☐ Display result in Hermes tab

Success:

Right-click request in Burp → Analyze with Hermes → See result in Burp tab.

Phase 9: Proxy History Integration

Goal:

Burp extension can scan Proxy history and update Hermes app map.

Tasks:

- ☐ Add button: Import Proxy History
- ☐ Respect Burp target scope
- ☐ Batch selected or in-scope proxy items
- ☐ Send to /proxy/import
- ☐ Display discovered domains
- ☐ Display likely API hosts
- ☐ Display high-value endpoints

Success:

Hermes shows the real frontend/API relationship map from Proxy traffic.

Phase 10: Repeater Integration

Goal:

Hermes helps create manual Repeater tests.

Tasks:

- ☐ Suggest Repeater mutations
- ☐ Let user choose which ones to create
- ☐ Create new Repeater tabs
- ☐ Label tabs clearly
- ☐ Save original and mutated request metadata
- ☐ Compare responses manually or semi-automatically

Success:

Hermes can turn a selected request into 3-5 guided Repeater test ideas.

Phase 11: Evidence Journal

Goal:

Hermes helps save proof for reports.

Tasks:

- ☐ Add evidence save endpoint
- ☐ Add Burp context menu: Save as evidence
- ☐ Link evidence to endpoint
- ☐ Link evidence to quest
- ☐ Export evidence as Markdown

Success:

Tester can save important request/response pairs with notes.

Phase 12: Custom Check / BCheck Generation

Goal:

Hermes helps convert confirmed manual findings into reusable checks.

Tasks:

- ☐ Add custom check generator
- ☐ Generate checks only from validated patterns
- ☐ Require tester review
- ☐ Mark generated checks as draft
- ☐ Store generated checks in project

Success:

Hermes can draft reusable checks from confirmed findings.

19. Safety and Scope Rules

Hermes must be safe by default.

Rules:

```
Passive observation by default
No automatic active testing unless explicitly approved
Respect Burp target scope
Separate in-scope and out-of-scope hosts
Do not send third-party traffic to LLM unless approved
Redact cookies, tokens, passwords, and secrets by default
Never claim a vuln without evidence
Use confidence levels
Do not generate destructive tests
Do not run tests against unknown domains
Keep a clear audit trail of suggested vs executed actions
```

Suggested redaction:

```
Authorization: Bearer [REDACTED]
Cookie: session=[REDACTED]
password=[REDACTED]
token=[REDACTED]
api_key=[REDACTED]
```

Allow the tester to toggle:

```
Send full headers to LLM: off by default
Send response bodies to LLM: configurable
Send third-party traffic to LLM: off by default
Import only in-scope Proxy traffic: on by default
```

20. First MVP Success Criteria

The first useful version succeeds if the tester can:

1. Paste 10–20 requests/responses.
2. Ask Hermes what the app seems to do.
3. See important domains and API hosts.
4. See important objects and workflows.
5. Get manual Repeater test suggestions.
6. Generate quests.
7. Track untested areas.

Example success output:

This appears to be a multi-tenant SaaS project management app.

Frontend:

- app.example.com

Likely first-party APIs:

- api.example.com
- auth.example.com

Important objects:

- project
- user
- membership
- invitation
- invoice

High-value tests:

- project IDOR
- tenant boundary bypass
- role escalation
- invite token abuse
- billing tampering

Untested:

- export endpoints
- admin routes discovered in JavaScript
- MFA challenge endpoint

21. What to Build First

Start here:

```
hermes-burp-assistant/  
  Dockerfile  
  docker-compose.yml  
  hermes_api/main.py  
  hermes_api/parser.py  
  hermes_api/analyzer.py  
  hermes_api/providers/mock.py  
  hermes_api/playbooks/broken_access_control.yaml  
  hermes_api/prompts/request_analyzer.txt
```

First endpoints:

```
GET /health
POST /analyze-request
GET /app-summary/{scope_name}
POST /generate-quests
```

Do not start with:

```
Burp extension
automatic scanning
BCheck generation
full AI agent autonomy
browser automation
local model inference requirements
```

Start simple:

```
Paste request + response
↓
Hermes explains endpoint
↓
Hermes updates app model
↓
Hermes suggests tests
↓
Hermes summarizes app
```

22. Long-Term Vision

Eventually Hermes becomes a Burp-native pentest navigator.

Future capabilities:

```
Live Proxy monitoring
Burp Site Map correlation
Repeater tab generation
Role/session comparison
Automatic response diffing
Quest tracking
Evidence export
Report note generation
Custom scan check drafting
```

- GraphQL schema understanding
- WebSocket message mapping
- Multi-user auth context tracking
- Frontend source map analysis
- Swagger/OpenAPI import
- Postman collection import
- Risk dashboard

But the core mission stays the same:

Help the tester understand the application and decide what to test next.